

Sarvah: Tech Stack & Rationale

Complete Technology Overview for Jury Questions & Team Alignment

1. Core Framework: Next.js 14 (App Router) + TypeScript

Why We Chose It:

- **Full-Stack Capabilities:** Next.js allows us to write both frontend React code and backend API routes (`/api/lots`, `/api/auth/register`) in a single unified codebase. This radically reduced our development time during the 4-day hackathon.
- **Server Components (RSC):** Next.js 14+ uses React Server Components, meaning we can fetch data securely on the server without sending heavy JavaScript bundles to the client. This is critical for users in rural areas with poor 3G/4G connectivity, as the pages load significantly faster.
- **SEO & Social Sharing:** Server-Side Rendering (SSR) ensures that if a farmer shares a lot link on WhatsApp, the preview cards will unfurl perfectly because the HTML is pre-rendered.
- **App Router:** Modern file-based routing with `app/` directory, giving us `layout.js`, `loading.js`, and parallel routing capabilities out of the box.

Benefits for Sarvah:

- Farmers in remote villages get fast page loads even on slow mobile networks
 - Buyers can share lot links on WhatsApp with proper preview cards
 - Single codebase for both farmer and buyer portals
 - Easy deployment on Vercel with zero-configuration
-

2. Language: TypeScript

Why We Chose It:

- **End-to-End Type Safety:** By defining strict interfaces (like `Lot`, `User`, `Transaction`), TypeScript catches bugs at compile time before the app even runs.
- **Data Integrity:** In an agricultural marketplace handling pricing and financial transactions, we cannot afford undefined variables or type coercion errors (e.g., treating a string `"500"` as a number `500` in a math calculation). TypeScript ensures strict numeric handling.
- **Refactoring Confidence:** When we need to scale or change backend logic, we can refactor with confidence that type errors will surface immediately.
- **Documentation:** Types serve as living documentation for the team - anyone can understand the data shape without reading implementation files.

Benefits for Sarvah:

- Zero runtime type errors in price calculations, quantity handling, and financial transactions
 - Self-documenting codebase - new team members understand data structures instantly
 - Catches bugs early in development, not at pitch time
 - Zod validation schemas share type definitions with runtime, ensuring consistency
-

3. Styling: Tailwind CSS v4 & Shadcn UI (Radix UI primitives)

Why We Chose It:

- **Rapid Prototyping:** Tailwind allows us to build a premium, highly responsive UI incredibly fast without managing messy external `.css` files. We shipped 15+ screens in 4 days.
- **Component Consistency:** We utilized Radix UI primitives (via Shadcn) to build accessible dropdowns, modals, and tabs. This ensures the app is fully usable by screen readers and operates smoothly on mobile browsers.
- **Responsive by Default:** Farmers will primarily access Sarvah on low-end Android smartphones. Tailwind's mobile-first breakpoints ensure the dashboard looks native on a 5-inch screen and scales beautifully to a desktop for institutional buyers.
- **Design System Consistency:** Using the same utility classes across all components ensures visual consistency without CSS specificity wars.

Benefits for Sarvah:

- Professional, polished UI without a dedicated designer
 - Fully accessible components (keyboard navigation, screen reader support)
 - Mobile-first approach works on the 5-7 inch screens farmers actually use
 - Rapid iteration - changing a color or spacing pattern is one line edit
-

4. Internationalization: next-intl

Why We Chose It:

- **First-Class Localization:** The problem statement requires serving Maharashtra's farmers. `next-intl` allowed us to build a robust Marathi-first interface where every single string is mapped to JSON dictionaries (`en.json` and `mr.json`).
- **Dynamic Routing:** It handles subpath routing automatically (e.g., `/mr/farmer` vs `/en/farmer`), making language context persistent across the user's session.
- **Framework Integration:** Built specifically for Next.js, so it plays perfectly with App Router and Server Components.
- **Marathi-First Approach:** Non-negotiable requirement - Marathi language takes priority in routing and UI display.

Benefits for Sarvah:

- 100% Marathi language support for target audience
 - English fallback ensures power users can also navigate
 - Language context persists across page loads and sessions
 - Meets the problem statement's language requirement fully
-

5. Security & Session Management: jose (JWT)

Why We Chose It:

- **Stateless Authentication:** We opted for JSON Web Tokens signed by the edge-compatible `jose` library. This means we don't have to constantly query a database to check if a user is logged in. The signed HTTP-only cookie contains everything we need.
- **Edge Compatibility:** Unlike heavier auth libraries, `jose` is ultra-lightweight and designed to run on Vercel Edge functions, ensuring lightning-fast middleware route protection.
- **HttpOnly Secure Cookies:** Prevents JavaScript XSS attacks from stealing user sessions.
- **7-Day Expiration:** Balanced security vs. convenience for farmers who may not log in daily.

Benefits for Sarvah:

- No database queries just to check authentication status
 - Fast middleware protection on all protected routes
 - XSS-resistant session management
 - Simple cookie-based flow that works on mobile browsers
-

6. Data Visualization: Recharts

Why We Chose It:

- **Declarative Charts:** Recharts integrates natively with React to build the 30-day Mandi Price Trend charts. It handles complex SVG rendering effortlessly.
- **Mobile Performance:** Performs well on mobile devices without crashing the browser - critical for farmer users.
- **Simple API:** `<LineChart><Line data={...} /></LineChart>` - intuitive for developers familiar with React.
- **Built-in Tooltips, Legends, Responsive Behavior:** Everything we need out of the box.

Benefits for Sarvah:

- 30-day price trend chart on farmer dashboard works flawlessly
 - No `d3.js` learning curve - simple React props
 - Charts resize correctly on mobile and desktop
 - Visual price data makes the SELL/WAIT recommendation actionable
-

7. State Management: SWR + nuqs

Why We Chose It:

- **SWR (Stale-While-Revalidate):** Fetch data on the server, keep stale copy client-side, revalidate in background. Perfect for price data that doesn't need instant consistency.
- **nuqs:** URL state management - query parameters automatically sync with component state, and vice versa. This means `/en/farmer?crop=soybean&district=latur` is automatically managed without manual state syncing.
- **Zero Configuration:** Both work out of the box with Next.js App Router.

Benefits for Sarvah:

- Price data loads fast, then updates in background
 - Shareable URLs with query params that actually work
 - Farmer can bookmark specific crop/district views
 - Seamless Marathi/English toggling via URL state
-

8. Validation: Zod

Why We Chose It:

- **Type-Safe Schema Validation:** Zod allows us to define validation schemas that are literally TypeScript types at runtime. We define once and get both validation and types for free.
- **API Route Safety:** All API routes (`/api/lots`, `/api/offers`, `/api/auth`) validate incoming payloads against strict schemas before processing.
- **Error Messages:** Zod generates human-readable error messages that we can display to users friendly.
- **Coercion Prevention:** Prevents malicious or malformed data from entering our system (e.g., negative quantities, price strings instead of numbers).

Benefits for Sarvah:

- Zero validation bypasses - every API endpoint is guarded
 - Consistent error handling across all routes
 - Runtime types match compile-time types
 - Fast development - validation schema = type + error messages
-

9. UI Component Library: shadcn/ui + Radix UI

Why We Chose It:

- **Accessibility First:** Radix UI primitives are the gold standard for accessible React components. Our dropdowns, modals, tabs, and buttons all work with keyboard navigation and screen readers.
- **Customizable:** We can override styles while keeping the solid accessibility foundation.
- **Premium Out-of-the-Box:** Components look professional without custom CSS - card, button, dialog, avatar, badge, etc.
- **Active Maintenance:** Backed by a large community and regular updates.

Benefits for Sarvah:

- Accessible marketplace for all users including disabilities
- Professional appearance without custom design work
- Consistent component library across farmer and buyer portals

- Mobile-touch friendly by default
-

10. Data Storage: JSON Files (Prototype Phase)

Why We Chose It:

- **Hackathon Agility:** To avoid the overhead of setting up, hosting, and migrating a heavy PostgreSQL database during a time-constrained hackathon, we built a robust JSON filesystem abstraction (`lib/data/store.ts`).
- **Future Migration Path:** Because we strictly typed our data access layer, replacing the `JSON.parse` logic with actual Prisma or Drizzle ORM calls to a PostgreSQL database in Phase 2 will take less than a day, without touching a single line of frontend UI code.
- **Zero Dev Ops:** No database servers, connections, or migrations to manage during the prototype phase.

Benefits for Sarvah:

- Shipping working prototype in hours, not days
 - No backend infrastructure costs during hackathon
 - Easy migration path documented and typed
 - Focus on frontend/business logic rather than database schema design
-

11. HTTP Client: SWR + fetch

Why We Chose It:

- **Stale-While-Revalidate:** SWR handles server fetching with built-in caching, refetching, and error handling.
- **Framework Integration:** Works perfectly with Next.js Data Fetching and Server Components.
- **Automatic Revalidation:** Background refetching keeps data fresh without manual polling.

Benefits for Sarvah:

- Price data stays fresh without manual polling code
 - Automatic caching reduces redundant API calls
 - Error handling and fallback states built-in
 - Works seamlessly with Next.js caching strategies
-

12. Date Handling: date-fns

Why We Chose It:

- **Modern Date Utilities:** `date-fns` is modular - we import only what we need (`convertSeconds`, `format`, `differenceInDays`, etc.).
- **Immutability:** All functions return new Date objects, no mutation surprises.
- **Timezone Handling:** Better than `Moment.js` for our use case of Mandi price dates.

Benefits for Sarvah:

- Reliable date calculations for 7-day trends, 30-day history, delivery windows
 - Format prices in Indian numbering system (₹1,00,000 format)
 - Calculate age of price data for recommendation engine
 - Minimal bundle size compared to alternatives
-

13. Icons: lucide-react

Why We Chose It:

- **Tree-Shakable:** Import only the icons you use, keeping bundle size minimal.
- **Consistent Design:** Single icon set across the entire application.
- **Easy SVG:** All icons are actual SVG, scalable and crisp at any size.

Benefits for Sarvah:

- Tiny icon bundle (only what we import)
 - Consistent visual language across all screens
 - No icon library bloat
-

14. Toast Notifications: sonner

Why We Chose It:

- **Fast & lightweight:** Sonner is the fastest toast library for React, built by the team that made Framer Motion.
- **Developer Experience:** Simple API: `toast.success("Offer accepted!")`
- **Animated:** Beautiful motion courtesy of Framer Motion integration.

Benefits for Sarvah:

- User feedback on transactions (offer accepted, offer rejected, transaction completed)
 - Non-blocking notifications that don't disrupt the farming workflow
 - Professional user feedback patterns
-

Summary: Tech Stack Rationale

Category	Technology	Primary Benefit
Framework	Next.js 14 + TypeScript	Full-stack unity, SSR/SSD, SEO
Language	TypeScript	Type safety, bug prevention
Styling	Tailwind CSS + Shadcn UI	Rapid prototyping, responsive mobile
i18n	next-intl	Marathi-first, dynamic routing
Auth	jose (JWT)	Stateless, edge-compatible security
Charts	Recharts	Mobile-friendly data viz
State	SWR + nuqs	Automatic sync, stale-while-revalidate
Validation	Zod	Type-safe, runtime validation
UI	shadcn/ui + Radix	Accessible, professional components
Storage	JSON files	Hackathon agility, easy migration
Dates	date-fns	Modular, immutable date ops
Icons	lucide-react	Tree-shakable, consistent
Notificationssonner		Lightweight, animated feedback

Overall Project Benefit: This stack allowed 3 developers to build a fully functional agricultural marketplace prototype in 4 days, with production-quality code, accessible UI, and a clear migration path to a robust database-backed system for Phase 2. The combination of Next.js + TypeScript + Tailwind enabled rapid development while maintaining code quality and scalability characteristics needed for the final product.

6. Jury Question Preparation Summary

Most Likely Questions & Prepped Answers:

- "Why Next.js over Create React App or CRA?"**
 - Next.js gives us Server Components for faster rural user pages, API routes in the same codebase, and automatic SEO - all critical for the problem statement.
- "How does authentication work without a database?"**
 - We use JWT signed cookies via the `jose` library. The cookie itself contains the user identity, signed with a secret key. No session database needed - the signature validates authenticity.
- "Why TypeScript for a hackathon prototype?"**
 - Catches type errors in price/quantity calculations before they become runtime bugs. Types serve as documentation. Migrating to a real database later is safer with strict types.
- "Why Marathi-first i18n?"**
 - The problem statement explicitly requires serving Maharashtra's farmers. Marathi language access is a non-negotiable requirement for the target audience.
- "How do you handle the price data - real or mock?"**
 - For the demo, we use seeded JSON data to guarantee performance. The architecture is designed to swap in real AGMARKNET API calls in Phase 2 with minimal code changes.
- "What's the 3.5% FPO premium and why?"**
 - When farmers pool their yield through FPOs, the system adds a 3.5% premium to the asking price. This structurally incentivizes bulk pooling for better leverage with buyers.
- "How scalable is the backend?"**
 - The JSON store is strictly typed - migrating to Prisma/Drizzle + PostgreSQL takes < 1 day. All TypeScript interfaces remain valid. API routes need minimal adjustments.
- "What's the recommendation engine logic in simple terms?"**
 - We look at: (1) Is current price in top 20% or bottom 30% of last 60 days? (2) Is price going up or down in the last 7 days? (3) Does the farmer have storage? (4) Do they need cash immediately? The combination of these factors determines SELL_NOW, WAIT, or HOLD.
- "How does the distance/freight calculation work without Google Maps API?"**
 - We use a pre-calculated district-to-distance matrix for the 5 key Maharashtra mandis. Freight rate is a simple heuristic (₹0.8 per km per quintal). Good enough for prototype; would integrate real API in Phase 2.
- "What's the biggest technical risk or limitation?"**
 - Mocked APIs (AGMARKNET, distance, verification) work for demo but would need real integrations for production. The JSON file storage would need to migrate to a relational database at scale. But both have clear migration paths documented.

File completed: This tech stack document covers all technologies used in the Sarvah project, why each was chosen, and the benefits they provide. It's designed to help the team answer any jury question confidently and explain the rationale behind technical choices.